How the handshake works

CR01-SC v2 | Technical preview 0.15.2

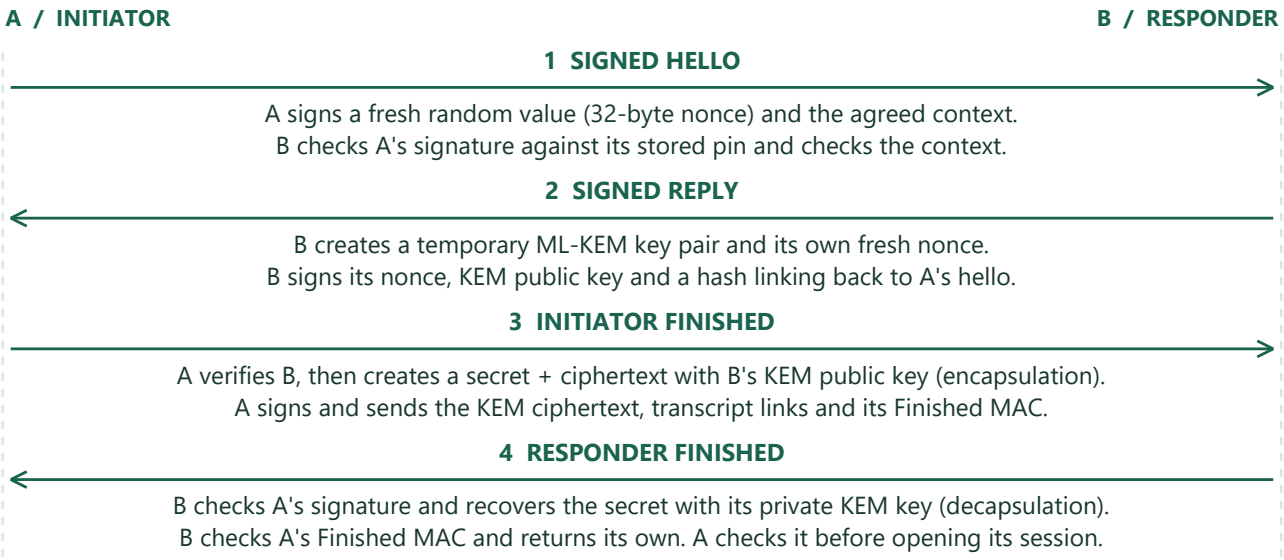
Two peers use trusted public keys to check each other's identity, establish a fresh shared secret and derive separate keys for each direction of communication.

THE ELEMENTS AND THEIR JOBS

ML-DSA-65	Signs the first three handshake messages. Trusted public-key pins identify the peer whose signature is checked.
ML-KEM-768	Establishes a fresh shared secret. A public KEM ciphertext is sent; the shared secret itself is not transmitted.
SHA-256 / HKDF / HMAC	SHA-256 hashes the exchange; HKDF-SHA256 derives separate confirmation and traffic keys. HMAC-SHA256 checks secret and transcript agreement.
AES-256-GCM	Encrypts data records and authenticates them, including the header and associated data.

Before starting: each peer has its own signing identity and an authenticated copy of the other peer's public signing key (a pin). Both agree on the application and policy epoch (configuration version).

FOUR MESSAGES, WITH A DIFFERENT CHECK AT EACH STEP



Signed context binds the suite/version, ordered peers and roles, application and epoch. The transcript is the ordered handshake record; hashes link the messages together.

Finished is a keyed transcript check, not a delivery acknowledgement. B opens locally when forming message 4; A opens after verifying it. If that last reply is lost, start a new handshake.

WHAT PROTECTS THE DATA AFTER SETUP?

The shared secret and final transcript feed HKDF-SHA256 to derive separate A-to-B and B-to-A traffic keys. AES-GCM authenticates each record's session, direction, sequence and associated data. Sequence counters supply nonces; a 64-record replay window rejects duplicates. Tag verification happens before plaintext release and replay-state commit. Handshake metadata remains public.

Research preview, not production-ready. Post-quantum primitives do not prove this custom protocol quantum-secure. No independent audit or side-channel assurance. Requires trusted pins, protected identities and trustworthy OS randomness. Restart needs a fresh handshake; no resumption or live-memory rollback detection. Cleansing is best effort, not process-wide secure memory.

PolyForm Noncommercial 1.0.0 | Commercial use is not granted.

Reference: docs/PROTOCOL.md